

Virtualização de Sistemas

Infraestruturas de Computação na Nuvem, Aula 03

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

Agenda (1/2)

- Conceito de virtualização e motivação.
- Os requisitos de Popek e Goldberg.
- Hypervisors tipo 1 vs tipo 2.
- Estratégias de virtualização de CPU.
- Os anéis de privilégio e o problema do x86.
- Trap-and-emulate: como o hypervisor intervém.

Agenda (2/2)

- Virtualização de memória, rede e armazenamento.
- Memory overcommit: KSM e ballooning.
- Virtualização de I/O: emulação, passthrough e SR-IOV.
- Formatos de disco virtual e thin provisioning.
- Snapshots e clones.
- Migração ao vivo e overhead de virtualização.
- KVM, LXC e a plataforma Proxmox VE.

Conceito de Virtualização e Motivação

- Virtualização: criação de uma versão abstrata (virtual) de um recurso físico, permitindo executar múltiplos sistemas isolados sobre o mesmo hardware.
- **Consolidação:** redução do número de máquinas físicas necessárias, melhorando o aproveitamento de recursos.
- **Isolamento:** falhas ou comprometimentos numa VM não afetam diretamente as restantes.
- **Portabilidade:** uma VM pode ser movida, copiada ou replicada independentemente do hardware subjacente.

Os Requisitos de Popek e Goldberg

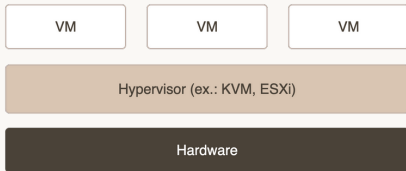
- Em 1974, Popek e Goldberg formalizaram três propriedades que um hypervisor deve garantir.
- **Equivalência:** um programa a correr na VM comporta-se como correria diretamente no hardware.
- **Controlo de recursos:** o hypervisor mantém controlo total sobre os recursos físicos atribuídos.
- **Eficiência:** a maioria das instruções executa diretamente no processador, sem intervenção do hypervisor.
- Estes critérios continuam a ser a referência para avaliar qualquer solução de virtualização.

Hypervisors Tipo 1 vs Tipo 2

- **Tipo 1 (bare-metal):** executa diretamente sobre o hardware, sem sistema operativo intermédio. Exemplos: VMware ESXi, KVM, Xen, Hyper-V.
- **Tipo 2 (hosted):** executa como uma aplicação sobre um sistema operativo anfitrião. Exemplos: VirtualBox, VMware Workstation.
- Hypervisors tipo 1 têm menor overhead e são a escolha habitual em datacenters de produção.
- Hypervisors tipo 2 são mais simples de instalar e adequados a ambientes de desenvolvimento e teste.

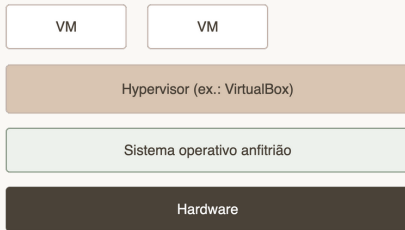
As Duas Arquiteturas de Hypervisor

Tipo 1 (bare-metal)



menos camadas, menor overhead
— usado em produção

Tipo 2 (hosted)



corre como aplicação sobre um SO
— usado em desenvolvimento

Estratégias de Virtualização de CPU

- **Virtualização completa:** o hypervisor emula totalmente o hardware; o sistema convidado corre sem modificações.
- **Paravirtualização:** o sistema convidado é adaptado para comunicar diretamente com o hypervisor, reduzindo overhead.
- **Virtualização assistida por hardware:** extensões de processador (Intel VT-x, AMD-V) que aceleram a execução de instruções privilegiadas.
- A maioria dos hypervisors modernos combina virtualização completa com suporte de hardware.

Anéis de Privilégio e o Problema do x86

- A arquitetura x86 define **anéis de privilégio**, do anel 0 (máximo privilégio, kernel) ao anel 3 (aplicações).
- Um sistema operativo convidado espera correr no anel 0, mas numa VM esse lugar pertence ao hypervisor.
- Problema histórico: no x86 clássico, certas instruções privilegiadas falhavam silenciosamente em vez de gerarem uma exceção capturável pelo hypervisor.
- Soluções: tradução binária dinâmica (reescrever instruções problemáticas em tempo de execução) ou paravirtualização.
- **Intel VT-x** e **AMD-V** resolveram o problema na raiz, criando um modo de execução dedicado ao hypervisor.

Trap-and-Emulate: Como o Hypervisor Intervém

- Popek e Goldberg classificam as instruções em três tipos: **seguras** (sem impacto global, executam livremente), **sensíveis** (accedem a hardware ou tabelas de páginas) e **privilegiadas** (geram uma exceção – *trap* – se executadas em modo utilizador).
- O código convidado corre como uma aplicação normal; quando emite uma instrução sensível, o processador gera um *trap* e devolve o controlo ao hypervisor.
- O hypervisor apanha o *trap*, emula o efeito da instrução (por exemplo, uma escrita em disco vira uma escrita num ficheiro) e devolve o controlo ao convidado, que não nota a interrupção.
- Este mecanismo aplica-se a múltiplos subsistemas: CPU e registos, unidade de gestão de memória (tabelas de páginas), controlador de interrupções e temporizadores, e dispositivos periféricos.

Virtualização de Memória e Rede (1/2)

- Virtualização de memória: cada VM tem o seu próprio espaço de endereçamento, gerido pelo hypervisor.
- **Shadow page tables:** o hypervisor mantém uma tabela de páginas própria, traduzindo endereços da VM para endereços físicos.
- **EPT** (Extended Page Tables): suporte de hardware que acelera esta tradução, eliminando parte do overhead das shadow page tables.

Virtualização de Memória e Rede (2/2)

- Virtualização de rede: um **vSwitch** (switch virtual) interliga as interfaces de rede virtuais das VMs entre si e com a rede física.
- Permite criar redes isoladas, VLANs virtuais e políticas de tráfego sem alterar o cablagem física.
- Virtualização de armazenamento: discos virtuais (ficheiros) que abstraem o armazenamento físico subjacente, permitindo snapshots e migração.

Memory Overcommit: KSM e Ballooning

- Num servidor partilhado por várias VMs, atribuir a cada uma a memória máxima "no papel" desperdiça RAM real – a maior parte de um SO fica em cache e buffers raramente usados.
- **KSM** (Kernel Samepage Merging): o hypervisor deteta páginas de memória idênticas entre VMs semelhantes (o mesmo binário, as mesmas bibliotecas) e funde-as numa só página física, com *copy-on-write* caso uma VM a modifique.
- **Ballooning**: um driver dentro do convidado "infla" um balão, reclamando memória que o SO convidado liberta (via *page cache*); o hypervisor reatribui essa memória a outras VMs sem que o convidado perceba que foi retirada.
- Ambos os mecanismos permitem *overcommit* de memória com segurança, mas dependem de suporte explícito na plataforma – no Proxmox, são opções configuráveis por VM

Formatos de Disco Virtual

- **raw**: imagem sem estrutura adicional; desempenho máximo, mas sem funcionalidades próprias.
- **qcow2** (QEMU Copy On Write 2): formato nativo do QEMU/KVM, com suporte a snapshots internos, compressão e alocação dinâmica.
- **VMDK** (VMware) e **VHD/VHDX** (Microsoft): formatos equivalentes noutros ecossistemas.
- **Thin provisioning**: o disco ocupa em armazenamento apenas o espaço efetivamente escrito, não o tamanho nominal declarado.
- Permite sobrecomprometer armazenamento (*overcommit*), à custa de exigir monitorização atenta do espaço real disponível.

Snapshots e Clones

- **Snapshot:** captura o estado de uma VM num instante (disco e, opcionalmente, memória), permitindo reverter mais tarde.
- Funciona tipicamente por *copy-on-write*: apenas os blocos alterados após o snapshot são efetivamente copiados.
- **Linked clone:** nova VM que partilha as camadas base do disco com a original, ocupando pouco espaço adicional.
- **Full clone:** cópia completa e independente, sem qualquer dependência da VM de origem.
- **Template:** VM convertida em modelo só de leitura, usada como base para criar novas máquinas de forma reprodutível.

Migração ao Vivo (1/2)

- Permite mover uma VM em execução de um host físico para outro, sem interrupção perceptível do serviço.
- Uma VM em execução é, no fundo, três coisas: **ficheiros** (discos, muitas vezes já acessíveis nos dois hosts via armazenamento partilhado), **configuração** (endereços de rede, descrição de hardware) e **contexto** (memória RAM e estado do CPU).
- Fases típicas: *setup* (o destino reserva recursos), *push* (o estado de memória é copiado enquanto a VM continua ativa na origem), *stop-and-copy* (pequena pausa para migrar o último estado e reconfigurar a rede) e *pull* (a VM arranca no destino, procurando dados que ainda faltem).
- Fundamental para balanceamento de carga entre hosts e para manutenção de hardware sem downtime.

Migração ao Vivo (2/2)

- **Stop-and-copy:** a VM pára, todo o estado é copiado, a VM arranca no destino. Simples, mas com downtime elevado.
- **Pre-copy** (a abordagem mais comum): as páginas de memória são copiadas iterativamente enquanto a VM corre na origem; só as páginas "suja" na última ronda exigem uma pausa curta antes de transferir o controlo.
- **Post-copy:** a VM arranca quase de imediato no destino, e as páginas em falta são pedidas à origem à medida que são acedidas – menor tempo até ao arranque, mas com risco de faltas de página se a rede for lenta.
- **Migração híbrida:** combina as duas anteriores, copiando a maior parte antecipadamente e recorrendo a *post-copy* apenas para o resto.

Overhead de Virtualização

- Toda a camada de abstração introduzida pelo hypervisor tem um custo de desempenho, ainda que reduzido com suporte de hardware.
- Cenários onde o bare-metal ainda é preferível: cargas de trabalho com requisitos extremos de latência ou I/O, como bases de dados de alto desempenho ou computação de alta performance (HPC).
- A decisão entre virtualizar ou não depende do compromisso entre flexibilidade operacional e desempenho máximo.

Virtualização de I/O: Emulação, Passthrough e SR-IOV

- I/O é tipicamente o maior gargalo da virtualização: ao contrário de cálculo puro (maioritariamente instruções seguras), acesso a dispositivos envolve com frequência instruções sensíveis, logo mais intervenção do hypervisor.
- **Emulação**: o acesso ao dispositivo gera um *trap*, o hypervisor emula o comportamento do periférico. Máxima compatibilidade, menor desempenho.
- **Passthrough** (I/O direto): um dispositivo físico (ou parte dele) é atribuído diretamente à VM; o convidado usa o driver nativo, com desempenho quase nativo – à custa de o host perder acesso a esse dispositivo.
- **SR-IOV** (Single Root IO Virtualization): extensão do PCI Express em que uma placa (rede, GPU, RAID) se apresenta como uma função física (gerida pelo host) e múltiplas funções virtuais, cada uma atribuível diretamente a uma VM diferente – o melhor dos dois mundos para hardware que o suporte

KVM: Virtualização no Kernel Linux

- **KVM** (Kernel-based Virtual Machine) é um módulo do kernel Linux que transforma o próprio kernel num hypervisor de tipo 1.
- Depende do suporte de hardware (Intel VT-x ou AMD-V) e integra-se com o **QEMU**, que trata da emulação de dispositivos.
- **virtio**: interface paravirtualizada para disco e rede, que evita emular hardware real e reduz significativamente o overhead de I/O.
- É a base tecnológica do Proxmox VE e de grande parte das clouds públicas assentes em Linux.

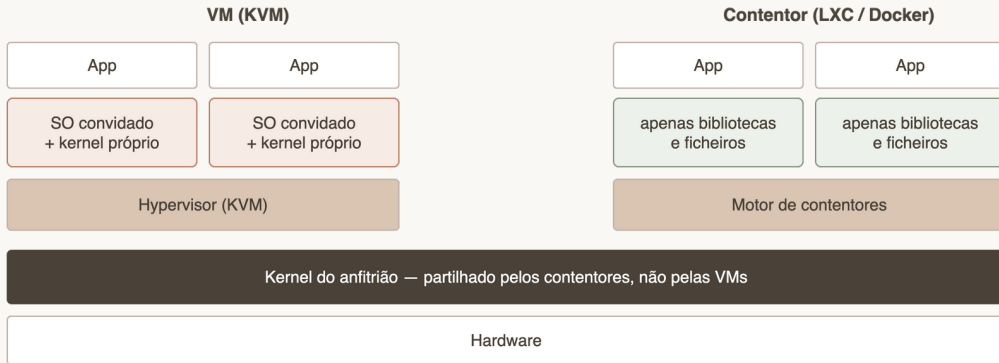
LXC: Virtualização ao Nível do SO

- **LXC** (Linux Containers) cria *contentores de sistema*: ambientes isolados que partilham o kernel do host.
- Não há hypervisor nem kernel convidado – daí o arranque quase instantâneo e o menor consumo de recursos.
- Assenta nas primitivas de isolamento do kernel Linux (namespaces e cgroups), que serão detalhadas na Aula 04.
- Limitação: só permite correr sistemas Linux, e o isolamento é mais fraco do que o de uma VM completa.

Proxmox VE: as Duas Tecnologias Lado a Lado

- O **Proxmox VE** é a plataforma de virtualização usada nos laboratórios desta UC.
- Integra numa única interface os dois modelos: **KVM** para VMs completas e **LXC** para contentores de sistema.
- Gestão por interface web ou por linha de comandos: `qm` para VMs KVM, `pct` para contentores LXC.
- Suporta snapshots, clones, templates e migração entre nós – todos os conceitos vistos nesta aula.
- É a plataforma que vai suportar o projeto prático da segunda metade do semestre.

O que Cada Modelo Duplica



É por isso que o contentor mostra o mesmo kernel do anfitrião e a VM não.

Síntese da Aula (1/2)

- A virtualização permite consolidação, isolamento e portabilidade de cargas de trabalho.
- Hypervisors tipo 1 (bare-metal) dominam em produção; tipo 2 (hosted) são úteis em desenvolvimento.
- Virtualização completa, paravirtualização e suporte de hardware (VT-x/AMD-V) são as três grandes estratégias de virtualização de CPU, todas a lidar com o mesmo problema: instruções sensíveis que o hypervisor tem de apanhar (trap) e emular.

Síntese da Aula (2/2)

- Memória, rede e armazenamento são também virtualizados, com mecanismos próprios (EPT, KSM/ballooning, vSwitch, discos qcow2/raw com thin provisioning).
- I/O pode ser emulado, passado diretamente à VM (passthrough) ou particionado com SR-IOV, consoante o compromisso entre compatibilidade e desempenho.
- Snapshots, clones e templates são as ferramentas operacionais do dia-a-dia; a migração ao vivo (pre-copy, post-copy ou híbrida) permite manutenção sem downtime.
- Existe sempre algum overhead; em cenários de desempenho extremo, o bare-metal continua a ser preferível.
- KVM (VMs) e LXC (contentores de sistema) coexistem no Proxmox VE, a

Obrigada.

Questões e comentários